Alphabets, Languages, and Grammars

Math 252

Terminology

- An **alphabet** (or **vocabulary**) is just a finite, non-empty set. It's elements are called **letters** or **symbols**.
- A **word** is a finite sequence of symbols.
- A word consisting of no symbols is called the **empty word** and deoted λ.
- The set of all words using symbols in alphabet V is denoted V^*.
- A **language** over V is a subset of V^*.

Exercises

1. Let $V = \{0, 1\}$. What is V^*?
2. Let $V = \{2\}$. What is V^*?
3. Can $\emptyset$ be a letter? an alphabet? a word? a language?

(Phrase-Structure) Grammars

A phase-structure grammar consists of

- **alphabet**: V
- **start symbol**: $S \in V$
- **terminal symbols**: $T \subset V$
  - $N = V - T$ is the set of **nonterminal symbols**
- **production rules**: $P \subseteq (V^* - N^*) \times V^*$
  - usually write elements of P as $u \to v$ rather than (u, v)
  - u must contain at least one nonterminal

Examples

Note: in each of these examples, capital letters are used for nonterminals and lower case letters or digits for terminals. That makes it easy to remember, but it is not a requirement of the definition of a grammar.

Grammar 1 (G_1): alphabet: $\{a, b, A, B, S\}$, terminals: $\{a, b\}$, start symbol: S, production rules:

- $S \to ABa$
- $A \to BB$
- $B \to ab$
- $AB \to b$

Grammar 2 (G_2): alphabet: $\{S, A, a, b\}$, terminals: $\{a, b\}$, start symbol: S, production rules:

- $S \to aA$
- $S \to b$
- $A \to aa$

Grammar 3 (G_3): alphabet: $\{S, 0, 1\}$, terminals: $\{0, 1\}$, start symbol: S, production rules:

- $S \to 11S$
- $S \to 0$

Grammar 4 (G_4): alphabet: $\{S, A, B, C, a, b, c\}$, terminals: $\{a, b, c\}$, start symbol: S, production rules:

- $S \to AB$
- $A \to Ca$
- $B \to Ba$
- $B \to Cb$
- $B \to b$
- $C \to cb$
- $C \to b$

Derivations and Languages

The rules of a grammar are used to derive strings of terminals (elements of T^*) as follows.

- If $w_0 \to w_1$ is a rule, then $lw_0r \Rightarrow lw_1r$ for any strings l and r.
- $a \overset{*}{\Rightarrow} b$ is defined recursively. $a \overset{*}{\Rightarrow} b$ if either
  - $a \Rightarrow b$, or
  - $\exists c\ (a \Rightarrow c \wedge c \overset{*}{\Rightarrow} b)$

Bascially the production rules are "rewrite rules" that allow us to replace the left side of the rule with the right side. A derivation is a sequence of rewrites.

The language of a grammar G is denoted $L(G)$ and contains all strings of terminals that can be derived from the start symbol:

$$L(G) = \{w \in T^* \mid S \overset{*}{\Rightarrow} w\}$$

Exercises

4. Show that using Grammar 1, $ABa \overset{*}{\Rightarrow} abababa$. Is $abababa \in L(G_1)$, the language generated by Grammar 1?
5. What is $L(G_2)$?
6. What is $L(G_3)$?
7. Create a grammar G_5 such that $L(G_5) = \{0^n1^n \mid n = 0, 1, 2, ...\}$
8. Create a grammar G_6 such that $L(G_6) = \{0^n1^n \mid n \in \mathbb{Z}^+\}$
9. Create a grammar G_7 such that $L(G_7) = \{0^m1^n \mid m, n \in \mathbb{N}\}$

Types of Grammars

Type 0: no restrictions

Type 1 (context sensitive): only two types of rules allowed

- $S \to \lambda$
  - shorthand: `[start]` $\to$ `[empty string]`
- $lAr \to lwr$ where
  - $l, r \in V^*$
  - $A \in N$
  - $w \in (V - \{S\})^+$ [That is, $w \neq \lambda$ and w doesn't use the start symbol.]
  - shorthand: `[left][nonterminal][right]` $\to$ `[left][non-empty without start symbol][right]`

Type 2 (context free): only one type of rule allowed

- $A \to w$ where
  - A is a single nonterminal symbol
  - $w \in V$ (so no restriction here)
- shorthand: `[nonterminal]` $\to$ `[anything]`

Type 3 (regular): Three types of rules allowed

- `[nonterminal]` $\to$ `[terminal] [non terminal]` (Example: $A \to bC$)
- `[nonterminal]` $\to$ `[terminal]` (Example: $A \to b$)
- `[start]` $\to$ `[empty string]` (Example: $S \to \lambda$)

A regular/context free/context sensitive language is a language that can be generated by a regular/context free/context sensitive grammar.

10. For each grammar we have seen so far, determine whether it is regular, context free, context sensitive. (Some grammars will be more than one. All grammars are type 0.)
11. True or false: Every grammar of type n is a grammar of type $n-1$.

Backus-Naur Form (BNF) for Context Free Grammars

Shorthand notation for type 2 grammars.

- nonterminals denoted with $\langle\rangle$.
- $\to$ written as `::=`
- All rules with same nonterminal on left written together with the list of possible right sides separated by `|`.

Example: If we have production rules $A \to Aa$, $A \to a$, and $A \to AB$, we will write

```
<A> ::= <A>a | a | <A><B>
```

BNF for ALGOL 60 identifier

```
<identifier> ::= <letter> | <identifier><letter> | <identifier><digit>
<letter> ::= a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

BNF for signed integer

```
<signed integer> ::= <sign><integer>
<sign> ::= + | -
<integer> ::= <digit> | <digit><integer>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

BNF specification for Java

You can find BNF for various programming lanugaes online. For example: https://users-cs.au.dk/amoeller/RegAut/JavaBNF.html has a BNF specification for Java.

Exercises

12. Why does BNF notation only work for context free grammars?
13. For each context free grammar we have seen, give its BNF representation.